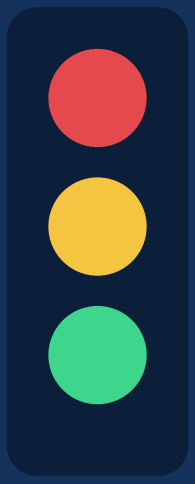
HACKATHON PROJECT REPORT

Intelligent Traffic Signal Controller

Using STM32F103RB



Domain	Embedded Systems and IoT - Intelligent Traffic Management
Microcontroller	STM32F103RB (ARM Cortex-M3)
Tools	STM32CubeMX, STM32CubeIDE, STM32 HAL, Embedded C
Peripherals	GPIO, TIM2, EXTI, NVIC, USART1

Requirements taken from Problem 6 of the supplied problem-statement PDF (Document2.pdf).

Table of Contents

Table of Contents 1
1. Project Title 2
2. Abstract 2
3. Introduction 2
4. Problem Statement 2
5. Objectives 2
6. Hardware Requirements 3
7. Software Requirements 3
8. System Architecture 4
9. Pin Configuration 4
10. Working Principle 5
11. Interrupt Handling and NVIC 6
12. STM32CubeMX Configuration Procedure 7
13. STM32CubeIDE Implementation 7
14. Proposed Algorithm 9
15. Testing and Validation Plan 9
16. Expected Results 10
17. Advantages 10
18. Limitations 10
19. Future Enhancements 10
20. Conclusion 11
21. Final Project Summary 11

1. Project Title

Intelligent Traffic Signal Controller Using STM32F103RB

2. Abstract

The Intelligent Traffic Signal Controller is an embedded system designed to manage traffic lights using a microcontroller. It controls red, yellow and green traffic signals according to a predefined sequence. A timer interrupt manages signal timing, while a pedestrian push button allows pedestrians to request a crossing. An IR sensor detects vehicle presence, and a relay represents an emergency signal. The system uses EXTI (External Interrupt) and NVIC (Nested Vectored Interrupt Controller) to handle interrupts with different priorities. UART communication transmits traffic-light status and system information. The project is configured using STM32CubeMX and developed using STM32CubeIDE.

3. Introduction

Traffic signals are essential for regulating vehicle movement and improving road safety. Traditional traffic signals often operate using fixed timing and may not respond directly to pedestrian requests or vehicle presence.

This project proposes a microcontroller-based traffic signal controller using the STM32F103RB. It combines timed traffic-light operation with pedestrian input, vehicle detection and emergency-signal control. It also demonstrates core embedded-system concepts: GPIO configuration, timer interrupts, external interrupts, interrupt priorities and UART communication.

4. Problem Statement

To develop an intelligent traffic signal controller that:

- Controls red, yellow and green LEDs.
- Implements a traffic-light state machine.
- Uses timer interrupts for signal timing.
- Supports pedestrian crossing through a push button and EXTI.
- Detects vehicle presence using an IR sensor.
- Controls a relay to represent an emergency signal.
- Uses NVIC priorities for pedestrian and emergency events.
- Transmits traffic-state information through UART.

These requirements are taken from Problem 6 of the supplied PDF (Document2.pdf).

5. Objectives

1. Develop an automatic traffic-light control system.
2. Control three traffic signals using STM32 GPIO pins.
3. Implement timer-based traffic-light timing.
4. Provide pedestrian crossing requests through a push button.
5. Detect vehicle presence using an IR sensor.

6 Represent emergency handling using a relay.
7 Configure EXTI and NVIC interrupt priorities.
8 Transmit traffic status through UART.
9 Understand embedded-system peripheral configuration using STM32CubeMX and STM32CubeIDE.

6. Hardware Requirements

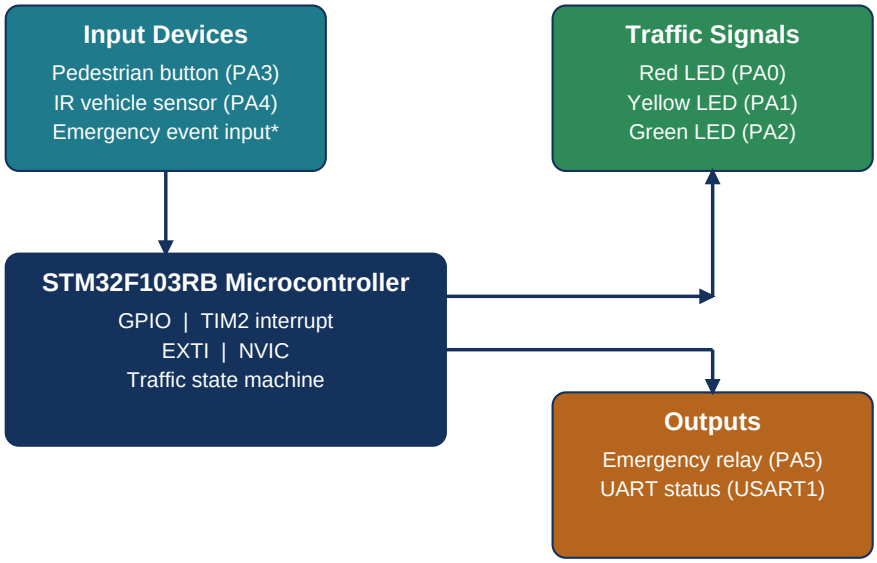
Component	Purpose
STM32F103RB microcontroller	Main controller
Red LED	Indicates stop
Yellow LED	Indicates caution
Green LED	Indicates go
Push button	Pedestrian crossing request
IR sensor	Vehicle presence detection
Relay module	Represents the emergency signal
LED resistors and connecting wires	Circuit connections
USB / ST-LINK programmer	Programming and debugging
External UART-to-USB adapter or serial interface	Viewing UART messages, if required

The LEDs, sensor, button and relay can be connected to a compatible development board or an external circuit. The relay must not be connected directly to a GPIO pin; use a suitable relay driver or relay module.

7. Software Requirements

Tool	Role
STM32CubeMX	Configures the microcontroller pins, timer, interrupts, clock and UART.
STM32CubeIDE	C programming environment: code compilation, debugging and project execution.
STM32 HAL Library	Provides functions for controlling GPIO, timers, UART and interrupts.
Embedded C	Implements the traffic-light logic and sensor handling.

8. System Architecture



** The emergency input must be defined in the final circuit or software; the relay itself is an output.*

Figure 1: System block diagram

The input devices provide information to the microcontroller. The controller processes these inputs, manages traffic-light states using a timer, handles interrupt requests, controls the outputs and sends status messages through UART.

> **Note:** The emergency event input must be defined in the final circuit or software. The relay itself is an output.

9. Pin Configuration

The following is the proposed pin mapping selected during CubeMX configuration.

STM32 pin	Function	Configuration
PA0	Red LED	GPIO Output
PA1	Yellow LED	GPIO Output
PA2	Green LED	GPIO Output
PA3	Pedestrian push button	GPIO_EXTI3
PA4	IR sensor	GPIO Input
PA5	Emergency relay	GPIO Output
PA9	UART transmit	USART1_TX
PA10	UART receive	USART1_RX
PA13	Debug data	SWDIO
PA14	Debug clock	SWCLK

PA13 and PA14 are retained for programming and debugging. PC13, the onboard blue push button shown in the CubeMX screenshot, can remain enabled if unused by the project.

> **Board note (if a NUCLEO-F103RB is used):** PA5 is also wired to the onboard user LED, and PA2/PA3 are routed to the ST-LINK virtual COM port. Check the board schematic and solder bridges before connecting external circuits to these pins. Because UART is on USART1 (PA9/PA10), an external UART-to-USB adapter is needed to view messages.

10. Working Principle

10.1 Traffic-light operation

The system follows a predefined sequence:

1. Green LED turns ON to allow traffic movement.
2. Yellow LED turns ON to indicate a transition.
3. Red LED turns ON to stop traffic.
4. The sequence repeats continuously.

Pedestrian request flag (EXTI3) and emergency condition are checked by the state logic

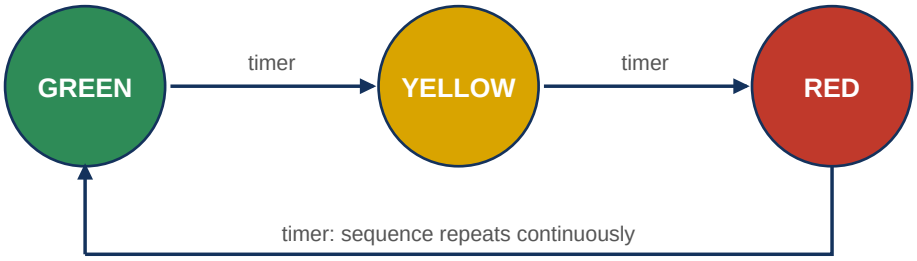


Figure 2: Traffic-light state machine

The state machine controls which LED is active at each stage. The exact timing values are selected during implementation.

10.2 Timer-based signal timing

TIM2 is configured as a basic timer using the internal clock. Its interrupt provides periodic timing events for the traffic-light sequence. The program counts timer events to determine when to switch between green, yellow and red states, avoiding reliance on software delay loops.

> **Illustrative timing example (assumption, not a project result):** With a 72 MHz timer clock, a prescaler of 7199 and an auto-reload period of 9999 give a 1 Hz update interrupt (72,000,000 / 7,200 / 10,000 = 1). The ISR can then count seconds, for example 10 s green, 3 s yellow and 8 s red. Final values depend on the verified clock configuration.

10.3 Pedestrian crossing

When the pedestrian presses the push button connected to PA3, the EXTI interrupt detects the event. The program records the crossing request and handles it through the traffic-control logic. The controller should only permit a safe crossing sequence after handling the current traffic state appropriately.

10.4 Vehicle detection

The IR sensor connected to PA4 provides a digital signal indicating vehicle presence. The microcontroller reads this signal and uses it in the traffic-control logic. The sensor's active-HIGH or active-LOW behavior must be verified on the actual module before finalizing the code.

10.5 Emergency signal

PA5 controls a relay that represents the emergency signal. When the defined emergency condition occurs, the controller activates the relay and executes the emergency-handling logic. The project must define how an emergency is triggered and how normal operation resumes. If a separate emergency input is required, an additional button or suitable input must be configured.

10.6 UART communication

USART1 is configured for serial communication. The controller can transmit the following messages, which help monitor and debug the system:

- Traffic: GREEN
- Traffic: YELLOW
- Traffic: RED
- Pedestrian crossing requested
- Vehicle detected
- Emergency signal active

11. Interrupt Handling and NVIC

The project uses two important interrupt mechanisms:

- **EXTI:** detects an external event, such as a pedestrian button press.
- **Timer interrupt:** generates periodic events for traffic-light timing.

The NVIC manages interrupt priorities. The proposed configuration gives EXTI3 a higher priority than TIM2:

Interrupt	Preemption priority	Subpriority
EXTI line 3	0	0
TIM2 global interrupt	1	0

A lower numerical priority value represents a higher priority in this configuration.

Open item: The emergency event must also be incorporated into the final design if the requirement is to assign priorities to both pedestrian and emergency events. A relay output alone does not generate an emergency interrupt.

12. STM32CubeMX Configuration Procedure

1. Create a new project for STM32F103RB.
2. Configure PA0, PA1 and PA2 as GPIO outputs.
3. Configure PA3 as GPIO_EXTI3.
4. Configure PA4 as GPIO input.
5. Configure PA5 as GPIO output.
6. Enable TIM2 with the internal clock.
7. Configure timer prescaler, counter period and counter mode according to the finalized clock.
8. Enable the TIM2 global interrupt.
9. Enable the EXTI line 3 interrupt.
10. Configure USART1 in asynchronous mode.
11. Assign the required NVIC priorities.
12. Verify the clock configuration.
13. Set STM32CubeIDE as the toolchain.
14. Generate the project code.

13. STM32CubeIDE Implementation

After generating the code in CubeMX:

1. Open the generated project in STM32CubeIDE.
2. Check the peripheral initialization functions in main.c.
3. Build the project to verify the generated configuration.
4. Start TIM2 using the HAL timer interrupt function.
5. Implement the traffic-light state machine.
6. Implement timer callback logic for changing states.
7. Implement the EXTI callback for the pedestrian button.
8. Read the IR sensor input.
9. Implement the emergency relay logic.
10. Transmit traffic status using UART.
11. Build, debug and test the application.

The generated interrupt handlers should be retained. Application logic should be added in the appropriate user-code sections so that CubeMX regeneration does not unnecessarily overwrite it.

Reference code sketch

The skeleton below is an illustrative outline of the intended structure (not a tested listing). Pin and handle names follow the CubeMX configuration above; timing values and emergency handling remain to be finalized.

```
/* USER CODE BEGIN PV */
typedef enum { ST_GREEN, ST_YELLOW, ST_RED } State_t;
volatile State_t state = ST_GREEN;
volatile uint8_t ped_request = 0, tick_flag = 0;
volatile uint32_t ticks = 0;
/* USER CODE END PV */

/* main(): after MX_..._Init() */
HAL_TIM_Base_Start_IT(&htim2);
while (1) {
  uint8_t vehicle = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_4); /* verify polarity */
  if (tick_flag) {
    tick_flag = 0;
    /* switch state when ticks reach the configured limit;
       honour ped_request at a safe point; set LEDs; send UART text */
  }
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
  if (htim->Instance == TIM2) { ticks++; tick_flag = 1; }
}
void HAL_GPIO_EXTI_Callback(uint16_t pin) {
  if (pin == GPIO_PIN_3) ped_request = 1;   /* keep ISR short */
}
```

14. Proposed Algorithm

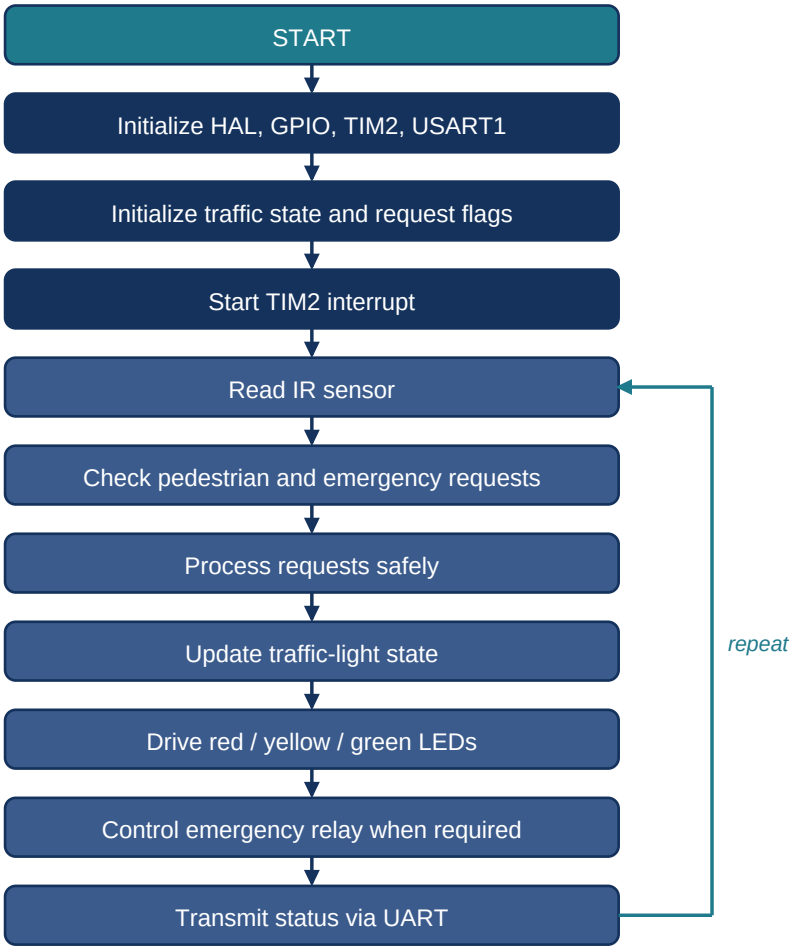


Figure 3: Control-loop flow

The timer interrupt maintains the timing sequence, and the EXTI callback records pedestrian requests. For reliable implementation, longer processing and UART transmission should generally remain outside interrupt callbacks.

15. Testing and Validation Plan

The following tests should be performed after implementation. They are **planned tests, not confirmed results**.

Test	Expected result
Power on the controller	Initial state is set safely
Normal traffic operation	LEDs follow the programmed sequence
Timer interrupt	Signal changes occur at the configured intervals
Pedestrian button press	EXTI detects the request
IR sensor activation	Vehicle presence is detected

Test	Expected result
Emergency event	Relay activates and emergency logic executes
UART communication	Correct status messages are received
NVIC priority check	Configured interrupt priorities are verified
Repeated operation	System continues to operate without unexpected state changes

16. Expected Results

After successful implementation and testing, the system should:

- Automatically cycle through the traffic-light states.
- Use TIM2 interrupts for signal timing.
- Detect pedestrian button presses through EXTI.
- Read vehicle presence from the IR sensor.
- Activate the emergency relay under the defined emergency condition.
- Use NVIC to manage interrupt priorities.
- Transmit traffic-state information through UART.

17. Advantages

- Demonstrates automatic traffic-light control.
- Integrates sensor input and user interaction.
- Demonstrates real-time interrupt handling.
- Uses modular peripheral configuration.
- Supports serial monitoring and debugging.
- Can be extended with additional sensors and traffic-management features.

18. Limitations

- The IR sensor detects only within its effective range and may be affected by environmental conditions.
- The prototype uses LEDs to represent traffic signals and does not directly control real road infrastructure.
- The relay represents an emergency signal unless connected to an appropriate external system.
- The basic state machine does not provide advanced traffic optimization or adaptive multi-road coordination.
- Real-world deployment would require additional safety mechanisms, fault handling and validation.

19. Future Enhancements

- Add multiple IR sensors to estimate traffic density.
- Implement adaptive signal timings based on vehicle presence.
- Add an audible pedestrian warning.

- Include a dedicated emergency input and a defined emergency override sequence.
- Display traffic status on an LCD or dashboard.
- Add wireless monitoring and remote status reporting.
- Extend the design to coordinate multiple intersections.

20. Conclusion

The Intelligent Traffic Signal Controller demonstrates how an STM32F103RB microcontroller can be used to implement a traffic-light control system with pedestrian interaction, vehicle detection and emergency-signal handling. STM32CubeMX simplifies the configuration of GPIO, timers, EXTI, NVIC and UART, while STM32CubeIDE supports embedded C programming, compilation and debugging.

The project provides practical experience with embedded systems, real-time event handling, peripheral configuration and microcontroller-based automation. Successful completion requires implementing the traffic state machine, integrating all inputs and outputs, and verifying the system through testing.

21. Final Project Summary

Intelligent Traffic Signal Controller Using STM32F103RB. The project develops an embedded traffic signal controller using the STM32F103RB, programmed through STM32CubeIDE and configured with STM32CubeMX. It uses GPIO-controlled LEDs to represent traffic lights, a timer interrupt to manage signal timing, an EXTI-enabled push button for pedestrian requests, an IR sensor for vehicle detection, a relay for emergency signaling, NVIC for interrupt priority management and UART for transmitting traffic status. The objective is to demonstrate a structured, interrupt-driven traffic-control system that can be tested as a hardware prototype.